Scheduling algorithms

First come first serve algorithm :

```
  GNU nano 8.7.1
#include <stdio.h>

int main() {
    int n, i;
    int bt[20], wt[20], tat[20];

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter burst time for each process:\n");

    for (i = 0; i < n; i++) {
        printf("P%d: ", i + 1);
        scanf("%d", &bt[i]);
    }

    // Waiting Time
    wt[0] = 0;

    for (i = 1; i < n; i++) {
        wt[i] = wt[i - 1] + bt[i - 1];
    }

    // Turnaround Time
    for (i = 0; i < n; i++) {
        tat[i] = wt[i] + bt[i];
    }

    // Display results
    printf("\nProcess\tBT\tWT\tTAT\n");

    for (i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\n",
               i + 1, bt[i], wt[i], tat[i]);
    }

    return 0;
}
```

Output:

```
userhasini@DESKTOP-QDSCCC4:~$ nano fcfs.c
userhasini@DESKTOP-QDSCCC4:~$ gcc fcfs.c -o fcfs
userhasini@DESKTOP-QDSCCC4:~$ ./fcfs
Enter number of processes: 3
Enter burst time for each process:
P1: 2
P2: 5
P3: 2

Process BT      WT      TAT
P1      2       0       2
P2      5       2       7
P3      2       7       9
```

Sjf non-premitive:

```
  GNU nano 8.7.1
#include <stdio.h>

int main() {
    int n, i, j;
    int bt[20], wt[20], tat[20];
    int temp;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter burst time:\n");

    for (i = 0; i < n; i++) {
        printf("P%d: ", i + 1);
        scanf("%d", &bt[i]);
    }

    // Sort burst times
    for (i = 0; i < n - 1; i++) {
        for (j = i + 1; j < n; j++) {
            if (bt[i] > bt[j]) {
                temp = bt[i];
                bt[i] = bt[j];
                bt[j] = temp;
            }
        }
    }

    // Waiting time
    wt[0] = 0;

    for (i = 1; i < n; i++) {
        wt[i] = wt[i - 1] + bt[i - 1];
    }

    // Turnaround time
    for (i = 0; i < n; i++) {
        tat[i] = wt[i] + bt[i];
    }

    printf("\nProcess\tBT\tWT\tTAT\n");

    for (i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\n",
               i + 1, bt[i], wt[i], tat[i]);
    }

    return 0;
}
```

```
userhasini@DESKTOP-QDSCCC4:~$ nano sjfnonpremitive.c
userhasini@DESKTOP-QDSCCC4:~$ gcc sjfnonpremitive.c -o sjfnonpremitive
userhasini@DESKTOP-QDSCCC4:~$ ./sjfnonpremitive
Enter number of processes: 2
Enter burst time:
P1: 3
P2: 4

Process BT      WT      TAT
P1      3       0       3
P2      4       3       7
```

Priority algorithm:

```
  GNU nano 8.7.1
#include <stdio.h>

int main() {
    int n, i, j;
    int bt[20], priority[20], wt[20], tat[20], p[20];
    int temp;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter burst time and priority:\n");

    for (i = 0; i < n; i++) {
        p[i] = i + 1;

        printf("P%d Burst Time: ", i + 1);
        scanf("%d", &bt[i]);

        printf("P%d Priority: ", i + 1);
        scanf("%d", &priority[i]);
    }

    // Sort according to priority
    for (i = 0; i < n - 1; i++) {
        for (j = i + 1; j < n; j++) {
            if (priority[i] > priority[j]) {

                // Swap priority
                temp = priority[i];
                priority[i] = priority[j];
                priority[j] = temp;

                // Swap burst time
                temp = bt[i];
                bt[i] = bt[j];
                bt[j] = temp;

                // Swap process number
                temp = p[i];
                p[i] = p[j];
                p[j] = temp;
            }
        }
    }

    // Waiting time
    wt[0] = 0;

    for (i = 1; i < n; i++) {
        wt[i] = wt[i - 1] + bt[i - 1];
    }

    // Turnaround time
    for (i = 0; i < n; i++) {
        tat[i] = wt[i] + bt[i];
    }

    // Display result
    printf("\nProcess\tBT\tPriority\tWT\tTAT\n");

    for (i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t\t%d\t%d\n",
               p[i], bt[i], priority[i], wt[i], tat[i]);
    }

    return 0;
}
```

Output:

```
userhasini@DESKTOP-QDSCCC4:~$ nano priority.c
userhasini@DESKTOP-QDSCCC4:~$ gcc priority.c -o priority
userhasini@DESKTOP-QDSCCC4:~$ ./priority
Enter number of processes: 4
Enter burst time and priority:
P1 Burst Time: 2
P1 Priority: 3
P2 Burst Time: 4
P2 Priority: 4
P3 Burst Time: 5
P3 Priority: 5
P4 Burst Time: 5
P4 Priority: 6

Process BT      Priority        WT      TAT
P1      2       3               0       2
P2      4       4               2       6
P3      5       5               6       11
P4      5       6               11      16
userhasini@DESKTOP-QDSCCC4:~$
```

Round robin algorithm:

```
  GNU nano 8.7.1
#include <stdio.h>

int main() {
    int n, i;
    int bt[20], rem_bt[20];
    int wt[20], tat[20];
    int quantum;
    int time = 0;
    int done;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter burst time:\n");

    for (i = 0; i < n; i++) {
        printf("P%d: ", i + 1);
        scanf("%d", &bt[i]);

        rem_bt[i] = bt[i];
        wt[i] = 0;
    }

    printf("Enter time quantum: ");
    scanf("%d", &quantum);

    while (1) {
        done = 1;

        for (i = 0; i < n; i++) {

            if (rem_bt[i] > 0) {
                done = 0;

                if (rem_bt[i] > quantum) {
                    time = time + quantum;
                    rem_bt[i] = rem_bt[i] - quantum;
                }
                else {
                    time = time + rem_bt[i];
                    wt[i] = time - bt[i];
                    rem_bt[i] = 0;
                }
            }
        }

        if (done == 1)
            break;
    }

    // Turnaround time
    for (i = 0; i < n; i++) {
        tat[i] = wt[i] + bt[i];
    }

    // Display results
    printf("\nProcess\tBT\tWT\tTAT\n");

    for (i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\n",
               i + 1, bt[i], wt[i], tat[i]);
    }

    return 0;
}
```

```
userhasini@DESKTOP-QDSCCC4:~$ gcc roundrobin.c -o roundrobin
userhasini@DESKTOP-QDSCCC4:~$ ./roundrobin
Enter number of processes: 3
Enter burst time:
P1: 12
P2: 6
P3: 14
Enter time quantum: 5

Process BT      WT      TAT
P1      12      16      28
P2      6       15      21
P3      14      18      32
```